



Bachelor Project Report

Interactive Signature Verification for Chia Blockchain

Alberto Centonze and Aidas Venckunas

Bachelor Students in Computer Science
École Polytechnique Fédérale de Lausanne

Prof. Rachid Guerraoui
Project Advisor

Dr. Pierre-Louis Roman
Project Supervisor

Distributed Computing Laboratory
School of Computer and Communication Sciences
École Polytechnique Fédérale de Lausanne

June 2023

Acronyms and abbreviations

BLS Boneh–Lynn–Shacham digital signature.

CLVM Chia Lisp Virtual Machine.

PKI Public Key Infrastructure.

PoST Proof of Space and Time.

List of Figures

1	The evolution of BLS constant-time verifiable signatures.	5
2	A standard transaction based protocol (a) compared to SIGINT (b, c)	8
3	The picture shows an extract from blockchain code where BLS signature validation is performed.	11
4	Transaction flow in Chia 1.8.1 without SIGINT	13
5	Diagram that shows how SIGINT is added as an optional possibility	14
6	Transaction flow in Chia 1.8.1 with SIGINT	14
7	Code redirecting transactions to the aggregator	14
8	The main aggregator loop	15
9	Dot-plot visualization of block verification time	18
10	Validation time as the number of coin grows.	19
11	Comparison of validation performances.	20
12	Validation performance in presence of stragglers.	21
13	Validation performance with digest generation.	22
14	SIGINT validation performance with a fraction of stragglers.	23
15	Validation time in a <i>perfect SIGINT scenario</i>	24

Contents

1	Introduction	5
1.1	Reasons for testing SIGINT performance in real-life systems	5
1.2	The Chia blockchain	6
1.3	Work phases and contribution	6
2	Background	7
2.1	SIGINT protocol	7
2.2	Clients	7
2.3	Aggregator	7
2.4	Validator	8
2.5	Chia blockchain processes	8
2.6	CLVM	8
2.7	Sending transactions and spending coins	9
2.8	Mempool	10
2.9	Block creation	10
2.10	Block Validation	10
2.11	Where SIGINT and Chia interleave	11
3	Implementation of SIGINT	12
3.1	Implementation details, simplifications and design choices	12
3.2	Overview of the aggregator	12
3.3	Overview of the changes in the Chia codebase	13
3.4	SIGINT transaction flow breakdown	13
3.4.1	Wallet synchronization and Aggregator connection	13
3.4.2	Pooling state	13
3.4.3	Technicalities of Chia's UTXO implementation	15
3.4.4	Digest Verification	15
3.4.5	Mempool forwarding	15
4	Evaluation of the hypothesis	16
4.1	Efficiency evaluation of current blockchain structure	16
4.2	Syncing	16
4.3	Metrics per block	16
4.4	Efficiency evaluation of isolated SIGINT protocol	18
4.5	Single signatures vs. fully aggregated multi-signatures	19
4.6	SIGINT and increasing number of stragglers	20
4.7	SIGINT and digest re-generation	21
4.8	Optimal block size	22
4.9	Modified Chia blockchain block validation metrics	23
5	Conclusion	25
5.1	Project Outcome	25
5.2	Final Considerations	25
5.3	Possible Future Improvements	25
5.4	Attributions	26

1 Introduction

Digital signatures are a fundamental cryptographic building block of the internet. In essence, a message signature enables a verifier to ensure both the (1) authenticity of the identity of the message signer and the (2) integrity of the signed message. While each signature is small and quick to verify, verifying even thousands of signatures takes a non-negligible amount of space and time, linear space and time to be exact. To circumvent this scalability challenge, cryptographers have built upon signature schemes such as **BLS** and Schnorr to propose mechanisms for signature aggregation that offer multiple benefits.

First, signature aggregation always saves **space**: an arbitrarily large number of signatures can be compacted into a signature of constant space, usually equivalent to the space required for just one signature. Second, signature aggregation can save **verification time**: akin to space, verification also takes a constant amount of time regardless of the number of signatures being verified, and is equivalent to verifying just one signature.

While aggregating several signatures always reduces the space footprint to a constant, it does not automatically translate into an equal reduction of the verification time. The latter ability depends on whether the signatures are: first, by the same signer for all messages, or, second, all on the same message. In the first case, aggregating **BLS** signatures from the same signer allows constant-time verification. In the second case, aggregating signatures on the same message and from different signers is referred to as multi-signatures as first conceptualized in 1983. Multi-signatures with constant-time verification - and with aggregatable public keys - have been proposed in 2018 for the **BLS** and Schnorr schemes.

SIGINT [1] is the first scheme that enables aggregate signatures to be both short and rapidly verifiable for different messages from different signers.

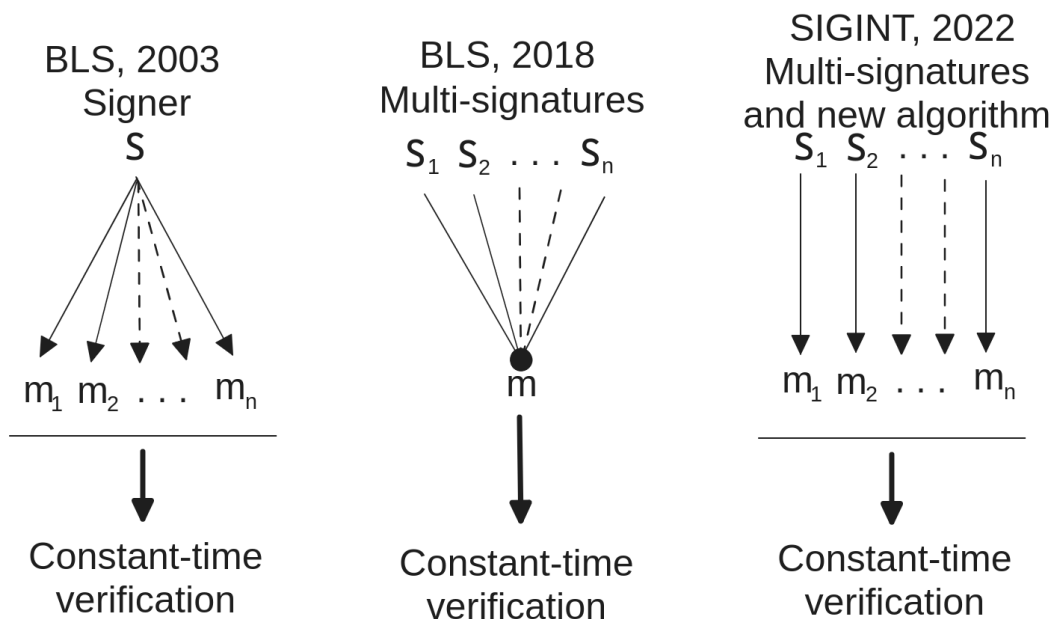


Figure 1: The evolution of **BLS** constant-time verifiable signatures.

1.1 Reasons for testing SIGINT performance in real-life systems

The evaluation of SIGINT performance in real-life systems is imperative due to the suggested interactive scheme's assertions of attaining constant time signature verification and maintaining a constant space

complexity. Conducting comprehensive tests within authentic operational contexts ensures that crucial factors, including potential bandwidth limitations, supplementary runtime needs, and computational costs, are adequately accounted for and taken into consideration.

Such comprehensive testing serves to establish the optimal operational parameters for the interactive scheme. Furthermore, implementing this scheme within existing systems enables a deeper understanding of the difficulty of structural modifications that are needed to seamlessly integrate and execute it within these specific environments.

One type of systems that are crucially dependent on cryptographic signatures and signature verification are blockchains. These systems are already at scale and since they are decentralized, it is feasible to run and test them without having to utilize immense amounts of computing power. Blockchains are also open-source, so any person can inspect the codebase structure of the blockchain and make customizations, program and change it to fit the needs. All of this lead to a decision to implement and run thorough testing of SIGINT verification scheme in a blockchain.

1.2 The Chia blockchain

Chia is a decentralized blockchain platform designed to provide a more energy-efficient and sustainable alternative to traditional cryptocurrencies. Chia introduces a unique consensus algorithm called **Proof of Space and Time (PoST)**, which leverages unused storage space on hard drives to secure the network. This approach ensures a more equitable distribution of mining power and reduces the reliance on energy-intensive mining processes.

The main reason why we chose to work with Chia blockchain is that it is one of very few blockchains to use **BLS** signatures for signature and block verification procedures. This capability significantly reduces our need to re-structure the codebase extensively and makes it relatively simple to run SIGINT scheme inside and outside of the blockchain compared to other blockchains.

1.3 Work phases and contribution

Our research was divided into two phases: implementation phase and benchmarking phase. During the first phase SIGINT interaction protocol was implemented and Chia Blockchain structure was adapted to work with SIGINT protocol. The second phase was focused on testing the signature validation performance in the current blockchain structure and evaluating performance that is achieved by using SIGINT protocol. These benchmarks then were compared in an extensive way.

The project was executed in a coordinated way where both students contributed to more complex tasks simultaneously. However, we concentrated and worked mainly on different phases: A. Centonze focused on the execution of implementation phase (section 3) and A. Venckunas (section 4) executed the benchmarking phase.

2 Background

Before we proceed to implementation and research details it is imperative to understand some key notions and processes that are present in Chia Blockchain and SIGINT interactive protocol.

2.1 SIGINT protocol

As mentioned in section 1, SIGINT protocol enables fast validation of cryptographic signatures. Its key structure consists of three entity groups that perform different actions and communicate between each other interactively: **clients**, **aggregator** and **validator**. The following sub-sections focus on each entity group to explain their role and behavior in SIGINT.

2.2 Clients

Clients are entities that send signed messages (payloads) to some system and these signed payloads need to be verified by a validator before submitting them to the system. In the context of a blockchain system, each user may be running the client (node) of the blockchain on its smartphone with the goal to submit the cryptocurrency transactions to the blockchain where before they are executed and added to the blockchain, these transactions need to be validated by a verifier that resides in a separate node of the blockchain. In SIGINT, clients send the signed messages to a proxy entity called aggregator instead of sending them to the validator. We will call these signatures $\sigma(m)$.

Clients that submit signed payloads then participate in an interactive signature exchange protocol with the aggregator that received the payloads. Hence, they have to stay online and ready to generate additional signatures when the aggregator requests them to do so. This protocol is explained in the next sub-section.

2.3 Aggregator

This entity is responsible for aggregating clients' payload signatures into one aggregated multi-signature that substitutes these multiple payload signatures. Aggregator (A) operates in two states: pooling state and aggregation state.

Firstly, A waits for incoming signed payloads from different clients (because of the way SIGINT works, the same signer cannot send multiple messages in the same batch), verifies each payload signature $\sigma_i(m_i)$ and adds it to a batch until it either has enough signatures $\sigma_1(m_1), \sigma_2(m_2) \dots \sigma_n(m_n)$ in a batch or some period of time has elapsed since last aggregation.

As soon as the batch is submitted for aggregation, A then computes the cryptographic commitment (e.g. using a Merkle tree) of the batch to obtain (1) the batch digest (e.g. the root of the Merkle tree) and (2) as many proofs of inclusion (e.g. using Merkle proofs of inclusion) as there are payloads in the batch. A prepares a request for each of the clients (1, 2, ..., n) with the expectation that they multi-sign the batch Merkle tree root using their **BLS** secret key, so that it is possible to aggregate them (as multi-signatures). Hence, A sends to each signer S_i the digest of the batch $digest(batch)$ and the proof that their payload is included in the batch $Proof(m_i \in batch)$.

When each client receives the digest and proof of inclusion, it verifies that (1) its payload m_i matches the received proof of inclusion $Proof(m_i \in batch)$, and that (2) the received $digest(batch)$ also matches the $Proof(m_i \in batch)$. If A sent correct digests and proofs of inclusions to all signers, then all signers received the same batch digest. Each client then generates its **BLS** multi-signature $\sigma_i(digest)$ by signing the batch digest and sends the pair $(\sigma(m_i), \sigma(digest))$ back to A thus attesting that their payload is in the batch whose digest they have signed.

A verifies that each multi-signature $\sigma_i(digest)$ it receives (1) is signing the correct batch digest computed by A and not another value, and that (2) it is indeed signed by client i using its public key PKI. A then aggregates all batch digest multi-signatures into a single $\Sigma_i(digest(batch))$. This aggregation ideally replaces all payload signatures in a batch with a single aggregated multi-signature if all clients correctly replied with their individual multi-signatures.

Slow or faulty clients (stragglers) may not send a multi-signature response fast enough when compared to other clients. In order to not penalize the fast signers, A cannot wait for all messages in forever. Therefore, regardless of the status or straggler clients, A proceeds to aggregate the digest multi-signatures it possesses after a given threshold time to ensure forward progress of the protocol for all. For straggler signers, A simply uses the payload signatures $\sigma_i(m)$ received initially to authenticate their payloads. The signatures from straggler signers are hence not aggregated. This is why A finally submits 2 signature types to the validator: (1) one aggregated multi-signature $\Sigma_i(\text{digest}(\text{batch}))$ where i is a fast signer (as opposed to straggler client) and (2) a group of single straggler signatures $\sigma_j(m_j)$ where j is a straggler signer. The aggregator then concludes its' aggregation state and falls back into the pooling state to listen and receive new transactions and form new batches.

2.4 Validator

Validator or verifier (V) is an entity that verifies all received payloads and submits verified ones into a system (or lets them through some pipeline inside the system) it is validating for. When a verifier receives all signatures by an aggregator, it needs to validate all of them.

First, V checks that no signer has more than one payload in the batch (for safety reasons). Secondly, it fetches from the **PKI** the **BLS** public key of each client who sent a multi-signature of the batch digest. V then (1) computes the aggregate public key PK using the **BLS** public key of each signer who **BLS** multi-signed the batch digest, and (2) computes the commitment of the batch to obtain the batch digest $\text{digest}(\text{batch})$. Using the computed PK and batch digest, V can verify for itself the correctness of the multi-signature $\Sigma_i(\text{digest}(\text{batch}))$. Verifying this aggregated multi-signature only requires a constant amount of time regardless of the number of signers as explained in introduction. Finally, V validates the correctness of the payload signatures from each straggler client once it fetched the corresponding public keys from the **PKI**.

There is also an optional acknowledgment mechanism that can be implemented if a system requires for it. If a batch multi-signature is verified, then aggregator is informed about the success and the success message is then propagated to multi-signature clients. Otherwise, the whole batch is invalidated and aggregator as well as all clients are informed about it.

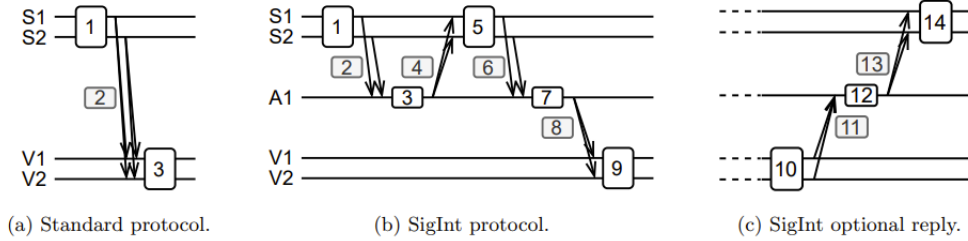


Figure 2: A standard transaction based protocol (a) compared to SIGINT (b, c)

2.5 Chia blockchain processes

In previous sub-sections we discussed how SIGINT protocol would operate in some abstract multi-entity system. Now we examine how Chia Blockchain operates to introduce its working principles and processes that conform to those that SIGINT is compatible with. After exploring these principles, it will be visible that Chia Blockchain system be adopted to implement SIGINT entities.

2.6 CLVM

The **Chia Lisp Virtual Machine (CLVM)** is a stack-based virtual machine that enables the development and execution of smart contracts on the Chia blockchain. It provides a secure, efficient, and deterministic

environment for implementing advanced transaction capabilities and decentralized applications within the Chia ecosystem.

More information about the **CLVM** can be found here: <https://chialisp.com/clvm/>

2.7 Sending transactions and spending coins

Chia blockchain as any other blockchain includes client nodes sending transactions to other client nodes residing somewhere in the blockchain's network. Chia uses coin set model, which has the following properties:

- The mantra **everything is a coin** will go a long way in helping you to understand the coin set model. There are no accounts. There are only coins.
- Technically the coins do not have owners. Anyone can attempt to spend any coin. Most coins are secured such that only someone who possesses a certain public/private key pair may spend them. This person is the coin's de facto "owner".
- The **coin set** is the total set of all coins on Chia's blockchain.
- The minimum value of a coin is 0 mojos. The theoretical maximum value of a coin is around 18 million XCH ($2^{64} - 1$ mojos). Each coin can be of any value within this range.
- Each coin can only be spent once. Thus, a coin has only two states: unspent and spent.
- A coin's value cannot be destroyed. Instead, when a coin is spent, its value is released. One or more new coins will be created in the same block, the total value of which will equal the value of the spent coin.
- Each coin has a puzzle associated with it that determines how, when, and by whom this coin can be spent. It must be chosen at the time of the coin's creation. For example, if Bob wanted to pay Alice, Bob would create a coin with a puzzle (and thus a puzzle hash) which Alice knows how to unlock. Bob can create a coin worth 5 XCH using Alice's puzzle hash, so that only Alice can unlock it.

When a client wants to spend its' coins, it creates a spend bundle that reveals the coins client will spend, the original puzzles, and the solutions to these puzzles. The solutions usually involves things such as conditions. Conditions always return a (**public key, message**) pair and require that a signature is present in order for the spend to be valid. In a standard transaction, this client is the only one that knows the private key that needs to be used to sign the message, and thus controls that coin. For each of these coin spends, puzzle conditions will indicate to sign them with different keys, so that a client does not sign any 2 messages with the same private key.

More information about coins can be found here: <https://docs.chia.net/coin-set-intro>

As mentioned in the paragraph above, a client creates a spend bundle when making a transaction. Spend bundle is essentially an aggregation of multiple coin spends and it is created when a transaction is being created by aggregating **BLS** signatures of all coin spends in a transaction (be aware that it is not an aggregated multi-signature and hence it is verifiable in linear time, but has constant space as explained in section 1). Multiple spend bundles can be combined (their aggregated signatures can be combined into bigger aggregated signatures), so they can represent a variable number of transactions. We can think about a spend bundle as a list of coin spends with their respective signatures aggregated into one.

More information on spend bundles can be found here: <https://docs.chia.net/spend-bundles>

2.8 Mempool

When a user makes a transaction, it gets sent to a full node, which then verifies it, adds it to the mempool (a queue of transactions), and broadcasts it to all of its peers. Therefore, transactions get propagated to the whole network in a very short period of time.

Only valid transactions are allowed to enter the mempool. The process of validating transactions is similar to the process of validating blocks and it includes running **CLVM**, checking conditions, validating signatures, and checking that the coins to be spent are currently unspent and valid.

The transaction is also checked against other transactions in the mempool, to ensure there are no conflicts.

More information on how the mempool works can be found here: <https://docs.chia.net/mempool>

2.9 Block creation

When the farmer makes a block, they will select the highest fee-per-cost transactions from the mempool until they reach the maximum block size. These spend bundles are combined into one large spend bundle, which is guaranteed to be valid, since all spend bundles in the mempool must spend disjointed coins. Coin spends cannot impact other coin spends, which is a property of UTXO systems that allows parallelization of validation and block creation.

During block creation, all spend bundles are aggregated into one big spend bundle (**list of coin spends, aggregated signature**). The aggregate signature is a combination of every signature from every transaction in that block. After the block is created, spend bundles no longer exist, the block contains a list of coin spends and one aggregated signature (which is not the multi-signature, but simple aggregated signature to achieve constant space complexity).

For performance reasons, the chia-blockchain codebase currently creates only smaller blocks (less than 50% of the maximum size) in order to keep the blockchain smaller and easier to run. This "throttle" is likely to be removed in future versions, after additional optimizations have been performed. To see how block sizes are computed, look at <https://docs.chia.net/coin-set-costs> . Each block size is different, but the maximum block size is approximately 400KB.

2.10 Block Validation

Block validation in Chia is composed of two parts: header validation and body validation.

The header validation performs consensus algorithm-related checks, such as **PoST**, signage points and infusion points, previous block hashes, foliage hashes, and timestamps. Notably, it does not validate any **CLVM**, coin spends, or signatures. Usually, for efficiency, light clients will want to validate headers but not the body.

Body validation entails running all puzzles for spent coins, reading the coin database, verifying signatures, checking for duplicate or invalid removals and additions, etc.

To validate the signatures, the validator extracts the list of coin spends and the aggregated signature as well as all messages that have been signed, fetches public keys from the coin puzzle conditions and **BLS** verifies the aggregated signature. If there are n coin spends in the block, then there are n public keys, and n messages, where size of each message is 96 bytes.

```

def aggregate_verify(
    pks: List[bytes48],
    msgs: Sequence[bytes],
    sig: G2Element,
    force_cache: bool = False,
    cache: LRUCache[bytes32, G2Element] = LOCAL_CACHE,
) -> bool:
    pairings: List[G2Element] = get_pairings(cache, pks, msgs, force_cache)
    if len(pairings) == 0:
        # Using AugSchemeMPL.aggregate_verify, so it's safe to use from_bytes_unchecked
        pks_objects: List[G1Element] = [G1Element.from_bytes_unchecked(pk) for pk in pks]
        res: bool = AugSchemeMPL.aggregate_verify(pks_objects, msgs, sig)
    return res

```

Figure 3: The picture shows an extract from blockchain code where **BLS** signature validation is performed.

The number of signatures that are contained in a block is in most cases bigger than the number of transactions in the block and so it introduces unnecessary verification complexity. If all coin spends in the same transaction would be bundled and signed by one master key instead of signing them separately one-by-one, validation would be more lightweight.

2.11 Where SIGINT and Chia interleave

Chia blockchain and SIGINT protocol have matching similarities which let us try to implement SIGINT into Chia blockchain and establish ties between them.

1. It can be observed that Chia blockchain has 2 entities that communicate with the help of mempool: (1) the client and (2) the farmer (which equals to validator in SIGINT terms). This allows to introduce a 3rd middle-man entity that is apparent in SIGINT - the aggregator.
2. Clients have to wait until their transactions are included in a block, so introducing minimal additional delay to interact with aggregator does not impact transaction execution time critically.
3. **BLS** signatures are used in Chia blockchain and so it is compatible with SIGINT protocol that scales using **BLS** signatures.
4. There are no duplicate public keys possible for one transaction, so a transaction can be batched.
5. Chia block contains one aggregated signature per block of transactions and SIGINT protocol also generates one aggregated multi-signature per batch.

These observations lead to the conclusion that it is structurally feasible to implement SIGINT in Chia blockchain. All detailed insights on how we are able to benefit from these similarities to define the implementation structure for SIGINT in Chia are presented in the following section.

3 Implementation of SIGINT

3.1 Implementation details, simplifications and design choices

- Since testing was done locally to accelerate the process multiple users were simulated by sending from the same light client wallet using different private keys to sign transactions. In a real case scenario the aggregator should store the RPC URL for each client that sends a transaction, so that the interactive part can be continued independently on each wallet node.
- Some of the cryptographic verifications (block validation in the aggregator, signature checks) were replaced by placeholder functions for the sake of time.
- The blockchain wallet node communicates with the aggregator through HTTP requests instead of the native SSL secured JSON-RPC system used by Chia. On the other hand when the aggregator needs to communicate back to the blockchain the native python class `WalletRpcApi`.
- For obvious reason the system has not been tested on the actual network but rather on the chia simulator, giving us full control over the network.
- The acknowledgment part of SIGINT has not been implemented since it is not in the scope of the project.

3.2 Overview of the aggregator

When an aggregator is started it spawns two independent processes: (1) an HTTP server to handle the interactive parts and (2) an aggregation loop that waits for enough transactions to be received to then initialize the interactive communications with the signers.

The server has the following two endpoints that should be used by wallet nodes to interact with the protocol:

- `queue_transaction` to send transaction that have to be aggregated instead of being directly sent to the mempool.
- `receive_signed_digest` to accept signed digests from users that verified that their transactions were correctly included in the batch.

The aggregation loop is instead handled by the following class:

```
class Aggregator:
    signed_transactions: List[TransactionRecord]
    batch_threshold: int
    batch: List[SpendBundle]
    digest: MerkleTree
    signed_digest: List[G2Element]
    single_transactions: List[SpendBundle]
    wallet: WalletRpcClient
```

All the transactions that should at some point be aggregated are kept in the `signed_transactions` list until the length of the list reaches the `batch_threshold`. When the threshold is reached `SpendBundles` are extracted from each `TransactionRecord` and if the operation is succesful they get added to the `batch` while their `CoinSpend`s get added to the `digest` merkle tree.

Once the aggregator has a digest ready to be sent, the aggregator starts sending requests for its signature to the participants of the protocol. Those that will manage to answer in time will have their signature be added to the `signed_digest` list while stragglers will be added to the `single_transaction` list from the initial `batch`.

3.3 Overview of the changes in the Chia codebase

To make the SIGINT scheme work some changes were required to the chia blockchain code. The release used as a base and to which the report refers to is 1.8.1 and can be found at <https://github.com/Chia-Network/chia-blockchain/tree/1.8.1>.

Most of the changes were implemented in the `WalletRpcApi.py` a file that exposes some JSON-RPC endpoints that can be used and consequently reflected in its python wrapper used by the aggregator `WalletRpcClient.py`.

The most significant additions were two new wallet endpoints that can only be called by aggregators:

- `sigint_sign_digest` to send requests to sign the aggregated digests discussed in section 2.3
- `sigint_send_aggregated_transactions` to send aggregated transaction to the mempool. However this choice was quite arbitrary and it's probably a bad idea in production as explained in section 3.4.5

3.4 SIGINT transaction flow breakdown

3.4.1 Wallet synchronization and Aggregator connection

Once a wallet node is synchronized with the network an aggregator instance can be connected to it by passing its rpc url. The aggregator in the test environment is automatically locally on the port 3000. In a hypothetical real case scenario since the aggregator can be hosted in a separate machine and a public url should be exposed and passed to the aggregator as an argument.

3.4.2 Pooling state

As soon as the two entities are interconnected transactions can start to be aggregated. To achieve so with a minimalist approach the user willing to aggregated just has to send a normal transaction to the network while adding the string "agg" in the memos of the transaction.

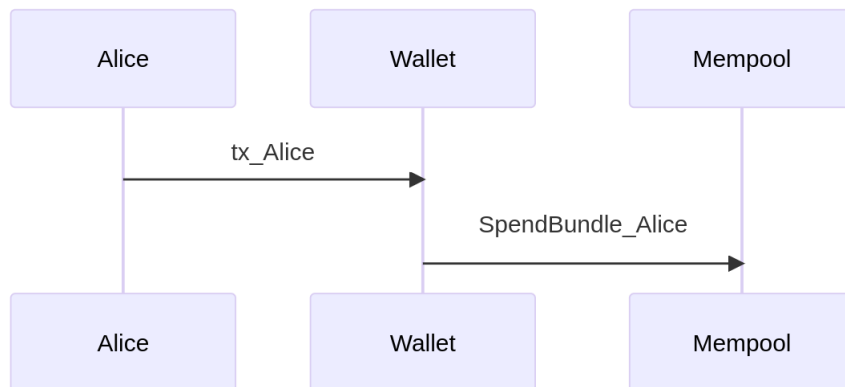


Figure 4: Transaction flow in Chia 1.8.1 without SIGINT

The node that receives the the transaction will look for the aforementioned specified string, if it's not present the transaction will be sent normally as shown in the figure above.

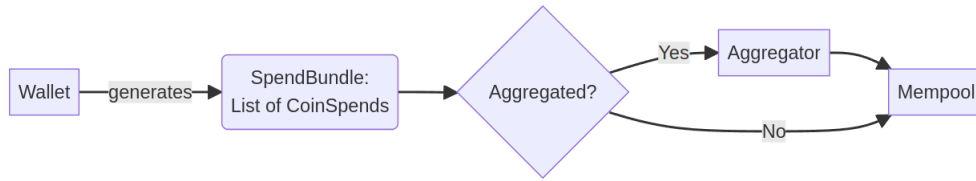


Figure 5: Diagram that shows how SIGINT is added as an optional possibility

If the aggregation memo is there, the node will create a SpendBundle as it would normally do but then instead of sending it to the mempool to include it in the block it will be sent back to the aggregator.

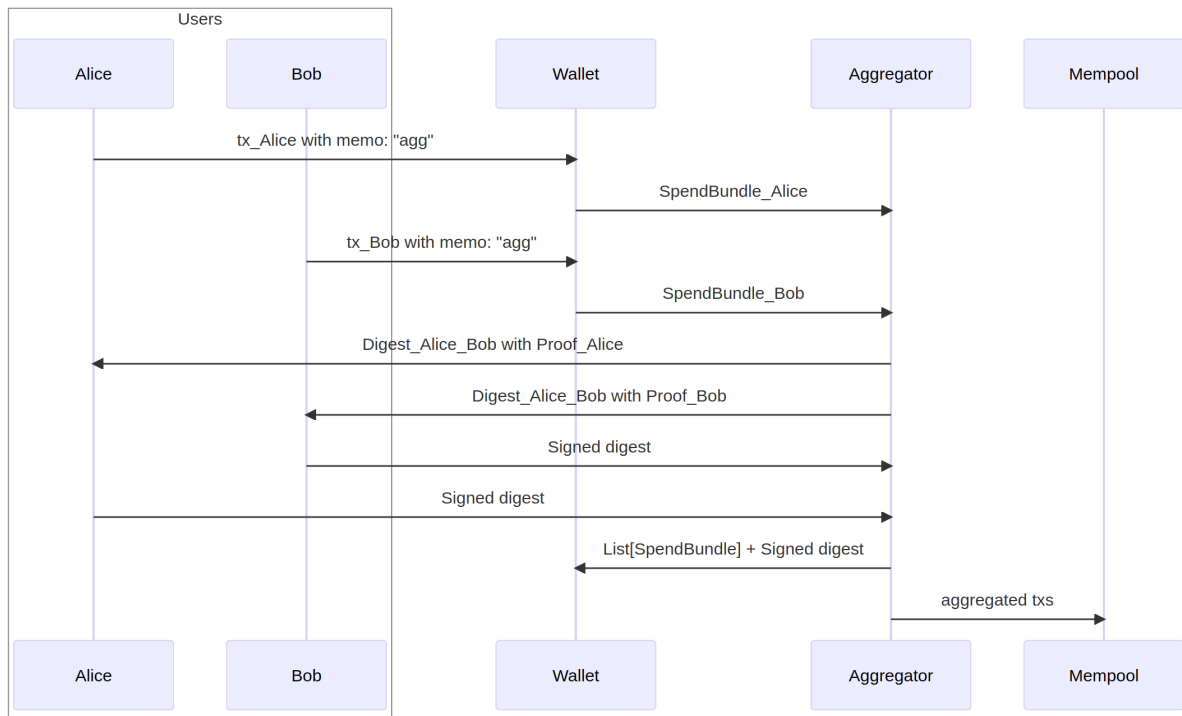


Figure 6: Transaction flow in Chia 1.8.1 with SIGINT

In the aggregation case a POST request is sent to the endpoint `queue_transaction` of the aggregator with the corresponding transaction (the spend bundle).

```

if "agg" in memos:
    post("http://127.0.0.1:3000/queue_transaction", json=tx.to_json_dict())
else:
    await wallet.push_transaction(tx)
  
```

Figure 7: Code redirecting transactions to the aggregator

Once the `SpendBundle` is received by the aggregator it is first validated as if it was received by the actual

network and then added to the next batch.

The main aggregator loop checks every two seconds (picked for testing purposes but should probably be faster in a real world scenario) and if the `batch_threshold` is reached it is finally possible to move to the aggregation state.

```
async def wait_for_txs(self):
    log.info("Started aggregator main loop")
    while True:
        await asyncio.sleep(2)
        log.info("Attempting aggregation")
        self.create_batch()
```

Figure 8: The main aggregator loop

3.4.3 Technicalities of Chia's UTXO implementation

The list of all the `CoinSpend` is extracted from the `SpendBundles` and is added to the digest merkle tree. The root and the proof is then sent to each wallet node so that they can verify the inclusion of their transaction.

Each wallet receives the digest and signs it once the verification is done. Then it proceeds to send the signature back.

Because of the way Chia works one transaction contains multiple `CoinSpend`s and each one of them is individually signed by a different public/private key pair derive from the user master key. In order to know which public key among the ones sent in the spend bundle has to be used to verify the digest signature an arbitrary decision had to be taken. In our case it was decided to pick the key of first `CoinSpend` to be used for the signature of the digest.

3.4.4 Digest Verification

When the aggregator receives the signature back it proceeds to fetch the public key of the first `CoinSpend` (obtained when the `SpendBundle` was initially received) and proceeds to verify the signature. If a straggler doesn't manage to send the transaction in time, the aggregator simply acts as a proxy and just forwards the transaction to the mempool.

3.4.5 Mempool forwarding

Once all the signatures are received and verified the transaction can finally be sent to the blockchain. The list of spend bundles that was initially created is sent (except for stragglers that are removed).

To achieve this an additional endpoint `sigint_send_aggregated_transactions` has been created for nodes to receive the transactions. At this point it's not clear who should be sending the transactions.

To simplify the implementation we just assumed that the aggregator can send it to the `sigint_send_aggregated_transaction` wallet endpoint without additional verifications. In a real world implementation the aggregator should be a special type of node with a private key to pay for gas and a slashing system to prevent misbehavior (delaying transactions).

4 Evaluation of the hypothesis

In order to test our hypothesis of SIGINT interactive signature validation protocol multiple testing phases were carried out. Initially, the current Chia blockchain performance was evaluated, where bottleneck verification points were observed. Then, isolated SIGINT protocol efficiency was benchmarked to stress-test these bottleneck points and verify that it provides a more efficient solution for verification. Optimal interactive protocol parameters were also established during this phase. Finally, adapted version of Chia blockchain that uses SIGINT multi-signatures was benchmarked to observe the changes in signature validation efficiency.

Due to the complexity of Chia blockchain and other limiting factors, some necessary simplification constraints were established. The testing was executed in a non-isolated environment that does not have dedicated hardware or any specialized software - a computer equipped with an SSD and an installation of Ubuntu 22.04 was used for this task. More in-depth set-up and constraint details for these phases as well as observed outcomes are discussed in the following sub-sections.

4.1 Efficiency evaluation of current blockchain structure

Initially, to be able to operate and execute transactions as an independent entity without a proxy, a full node client has to be initiated and synced with the blockchain. This requires the client node to bootstrap a chain of blocks which is an essential data structure that stores all crucial information about the history and state of the blockchain. To perform syncing, blocks are fetched in batches from peer nodes that reside in the blockchain network and contain all blocks up to the current one. Upon fetching each batch, the node then performs the block validation procedure block-by-block as explained in section 2.10. BLS signature verification needs to be performed during this procedure requiring the node to perform the following:

1. Acquire k (public key, message) pairs, where k is the number of coin spends in the block. These pairs are stored in the block as a separate data structure (Conditions). Each BLS public key is called a *G1Element* and has a size of 48 bytes. Each message as generated by a CLVM puzzle is 96 bytes in size.
2. Acquire an aggregated signature that encapsulates all signatures from each of the coin spend in a block. This signature is also stored in the block and is called a *G2Element* in BLS cryptographical terms. It has a size of 96 bytes.
3. Perform BLS *aggregate-verify* (as shown in 2.10) procedure to validate the aggregated signature by supplying the elements acquired in parts 1-2: signature itself, the list of public keys ($k \times 48$ bytes in total) and the list of messages ($k \times 96$ bytes in total).

This procedure is an important component of overall block validation time and it is the reason why it was decided to measure how critical it is in current Chia blockchain.

4.2 Syncing

A full node syncing operation was performed and it took **26 hours and 21 minutes** to sync **1017931** out of **3775228** blocks that were in the blockchain at the time of syncing (07.06.2023) in total. Due to the complexity and average size of a block increasing on a block-by-block basis it was decided not to continue until the node would be fully synced. Chia developer team confirmed that on 07.06.2023 the overall blockchain size was **113 GB** and average full sync time on a high-bandwidth machine was around **3 days**.

4.3 Metrics per block

Average block size in terms of coins spent in that block was calculated in the following way:

1. In Chia blockchain, about 40% of the blocks are left empty for consensus purposes. <https://docs.chia.net/consensus-foilage#transaction-block-time> These blocks are marked as non-transaction blocks in Chia and they were not included in the calculation
2. First **270000 blocks** did not contain any transactions, so there were no coins spent in them. It was because for some time after the blockchain launched, Chia did not support sending transactions. These initial blocks were not included in the calculation.
3. All the blocks that were not excluded in steps 1-2 were measured and their average coin spend count was calculated.

The final measure is **36.5 coins spent per block** with a maximal number of **890** coins spent per block.

Average block size in bytes is **28800** bytes with the maximum accepted size of **400KB**. More information on how the block size depends on different factors and how it is calculated is presented in <https://docs.chia.net/coin-set-costs>. Since the main focus of interest for this project was in signature validation time, this measure is not as important as the previous measure of average coins per block, since it does not impact the signature validation time directly.

Average block **signatures** validation time was calculated in the following way:

1. Empty blocks: empty transaction blocks and non-transaction blocks do not undergo the signature validation, so the time spent to validate signatures for these blocks is equal to 0. All empty blocks were therefore not included in the calculation.
2. All non-empty blocks were considered in average time calculation. The time spent for signature validation is equal to the time that **BLS** aggregate verify function took to execute.

The result on the measure of time spent per block to validate the signatures is **0.167 seconds** per block with a maximal time of **6.02** seconds per block.

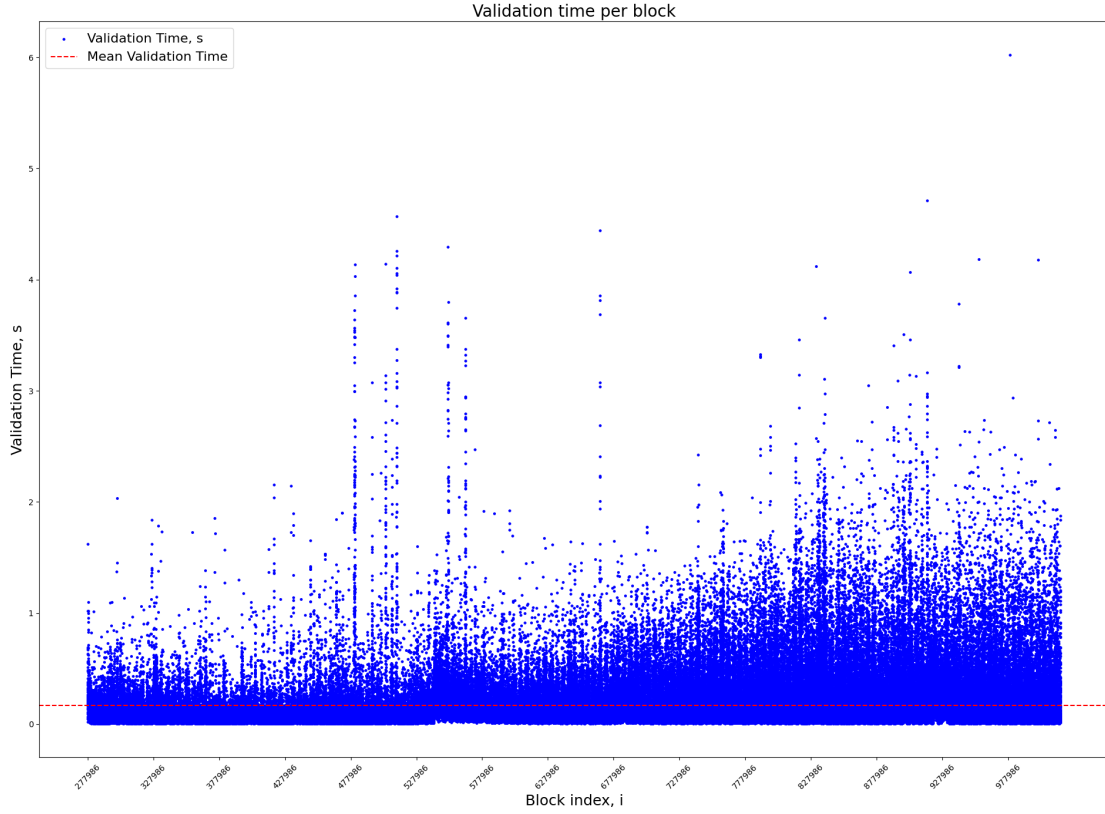


Figure 9: Dot-plot visualization of block verification time

These blocks were created between **May 3rd 2021** and **17th of October 2021**. It can be visible that earlier block signatures took less time to be validated, so there were fewer coins in these blocks. Over the time Chia blockchain got more popular and hence there were more transactions and consequently more coins per block.

4.4 Efficiency evaluation of isolated SIGINT protocol

After the SIGINT protocol system was created and the codebase was written, it was decided to initially test the system in an isolated environment, where the validator and all other participating entities reside outside of the blockchain, in minimal client-server threads. This was an important step before running the SIGINT system inside of Chia blockchain, since it enabled to find and finetune the optimal parameters that would later be used during execution in Chia.

For this specific testing purpose a dedicated benchmarking system was created and different range of tests were executed to comprehend the behavior of SIGINT under different assumptions. In the subsequent sections we will cover in detail each of the tests that were performed. All of the tests were ran **three times** to minimize the noise in resulting data.

Important detail to mention is that all the data-structures of the Chia blockchain such as coin spends and messages were used throughout these benchmarks, since (1) these data-structures have consistent size and therefore are comparable between each instance and (2) it was crucial for finetuning parameters, since the same data-structures would be used when executing SIGINT inside of Chia blockchain.

4.5 Single signatures vs. fully aggregated multi-signatures

First benchmarking phase was performed by comparing the primitives: single **BLS** signatures (or simple aggregated signatures) and **BLS** multi-signatures. It comprised of running **BLS** validation for the 2 types of signatures in an increasing amount of coin signatures that comprised the signature.

For single signatures, the benchmark started with measuring validation time for one coin and respective signature $\sigma_i(m_i)$, increasing the number of total signatures each iteration until it reached **890 signatures** in total - the maximal number of coins observed in one block during the blockchain syncing.

As initially expected, this operation has time complexity **$O(n)$** where n is the number of signers. The following figure is a graph that represents exact running times which take up to **1.32 seconds** per validation.

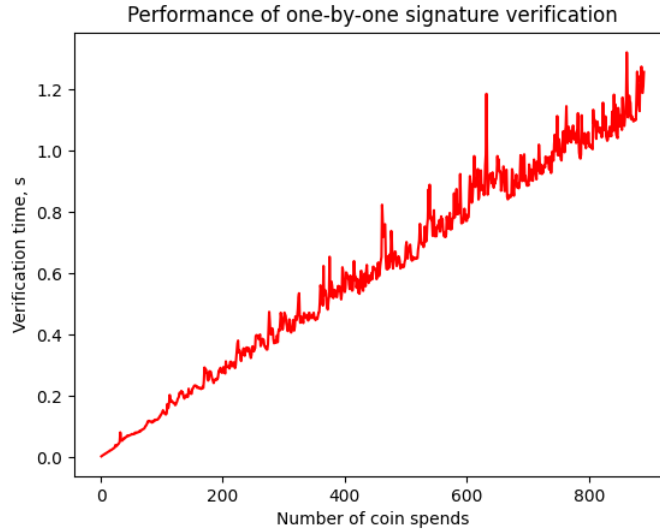


Figure 10: Validation time as the number of coin grows.

For aggregated multi-signatures the benchmark was executed in the same manner as for single signatures, starting from a multi-signature that comprised of one coin spend signature and increasing the number of coin spends that comprise aggregated multi-signature each iteration until the same number (890) of coin spends comprising aggregated multi-signature was reached. As a reminder, in this case multi-signatures were used, hence the message being signed was always unique and instead of i different messages in iteration i , there was only one message (the root of a digest) that was signed by i different actors to create an aggregated multi-signature. In this test, the time to re-generate digest for multi-signatures was not included in the overall time measure. However, it was included in subsequent testing phases where it is mentioned.

By definition of **BLS** multi-signature verification, a multi-signature is always validated in time **$O(1)$** compared to **$O(n)$** earlier. The following figure is a graph that represents exact running times which take up to **0.006 seconds** per validation.

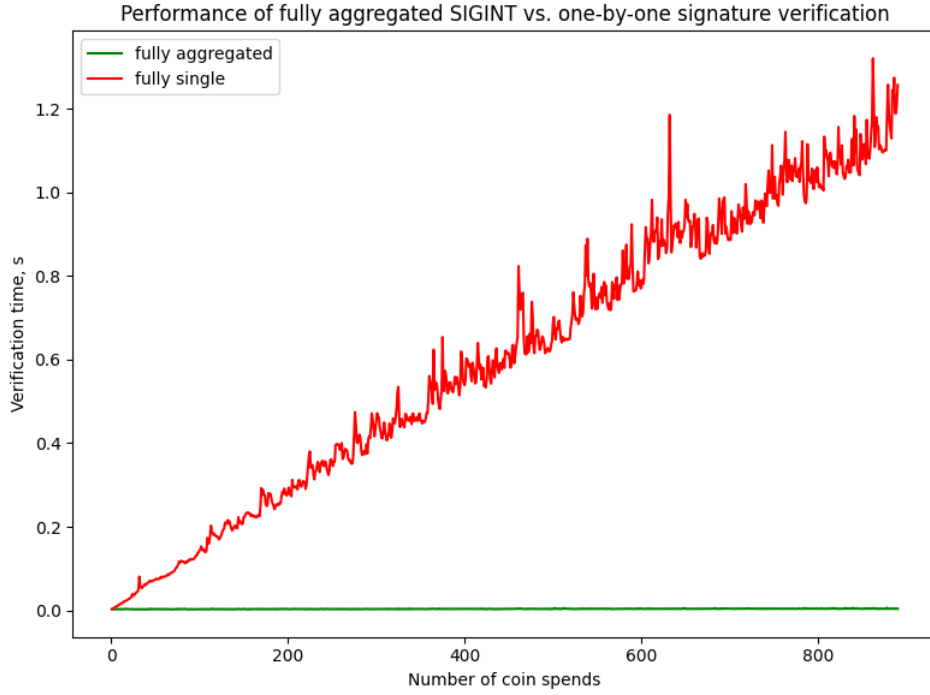


Figure 11: Comparison of validation performances.

4.6 SIGINT and increasing number of stragglers

In the fully aggregated multi-signatures benchmark a sharp approximation was that all signers are fast signers and there are no straggler clients. However, in a real-world scenario it is not the case. Hence, this second testing phase measures how a validation of a large block of 890 transactions is impacted by an increasing number of stragglers. In this case the validation is executed in 2 steps:

1. The batch of coin signatures that were made the fast signers and have an aggregated multi-signature is validated with **BLS** multi-signature verify function.
2. All straggler coin signatures are verified one-by-one by **BLS** signature verify function.

The benchmark computed the overall time to execute both steps for each iteration and resulted in a linear increase in validation time with an increasing number of stragglers. This is indicated in the following figure below.

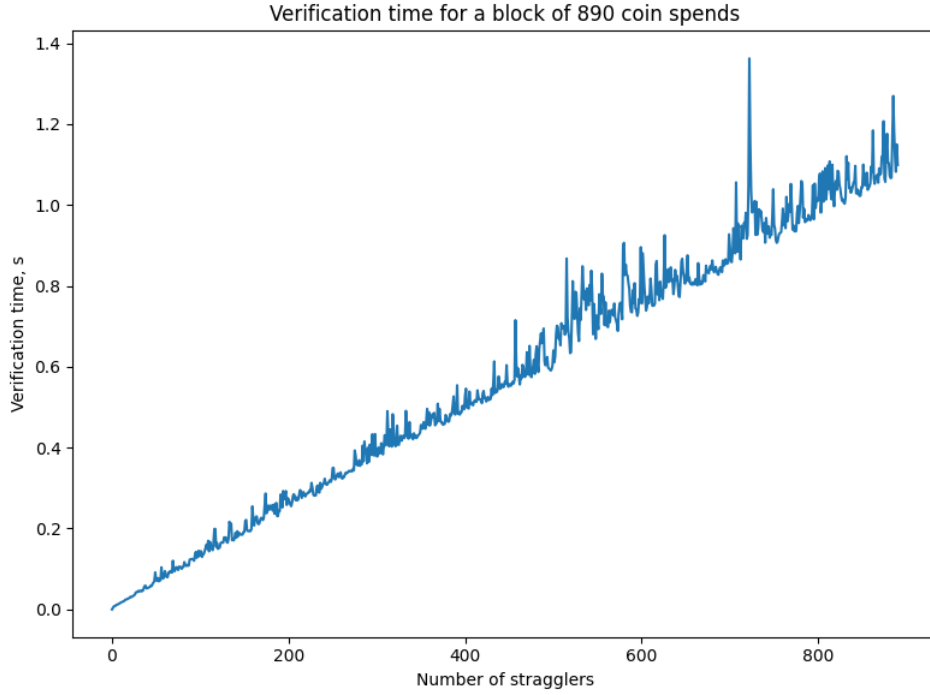


Figure 12: Validation performance in presence of stragglers.

This result is completely valid and is sharply equal to the one-by-one signature verification, the running time is linear in the number of stragglers. However, it can be observed that there is apparent noise that always results in an uptick of running time. This noise is more sharp starting from around **280 stragglers**. We believe that this result can be motivated by hardware memory accesses needed when data becomes bigger.

4.7 SIGINT and digest re-generation

One simplification that was presented during the benchmarking of validation time for fully aggregated multi-signatures evaded including the time to re-generate a digest when the signature verification is performed. However, in real-life systems it is imperative to re-generate the digest each time an aggregated multi-signature is being verified as suggested by SIGINT in section 2.4. It is also expected to take longer than constant time with an increasing number of signers in the aggregated multi-signature. Hence, the same test as in phase one was performed, but this time the time to re-generate the digest was included in the final computation. The following figure is a graph that shows what impact it has to overall verification time.

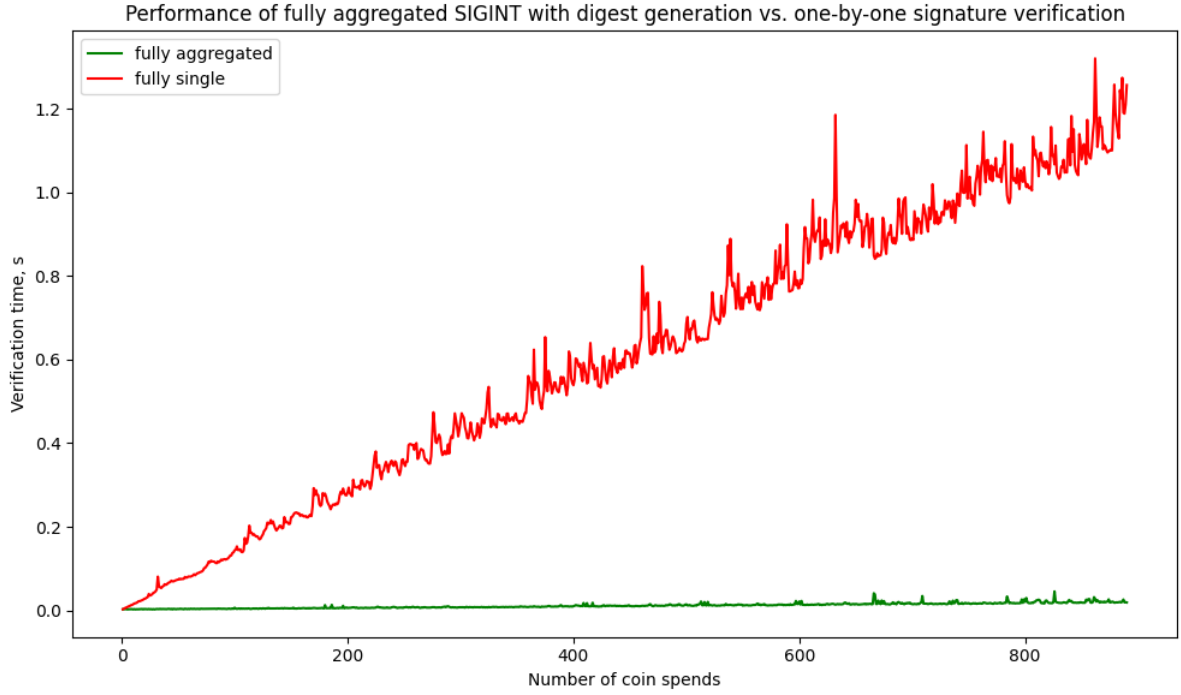


Figure 13: Validation performance with digest generation.

One unfortunate impact is that the time complexity is not $\mathbf{O(1)}$ anymore, but it becomes sublinear with an extremely small constant. However, it is not a limiting problem, since compared to single signature verification times, the aggregated multi-signature still performs incomparably better with running times ranging up to **0.045 seconds** per validation vs. up to **0.006 seconds** without digest re-generation.

4.8 Optimal block size

Final testing piece for isolated SIGINT protocol is choosing the optimal batch (or block) size as a number of coin spends in it given some threshold fraction of straggler coin spends. It was chosen to accept up to 10% of stragglers, since (1) it is the amount at which the least up-ticking noise is apparent and (2) we want our system to aggregate as much as possible, leaving only the slowest signers behind.

The benchmark consisted of i iterations where each iteration i represents SIGINT validation time for a block of i coin spends and **10%** stragglers. Overall verification time was calculated in the same two-step process as in testing section 4.6 except that it also included digest re-generation impact. The following figure is a graph showing how long it take for different sized blocks to be verified and it ranges up to a block of **890 coin spends**.

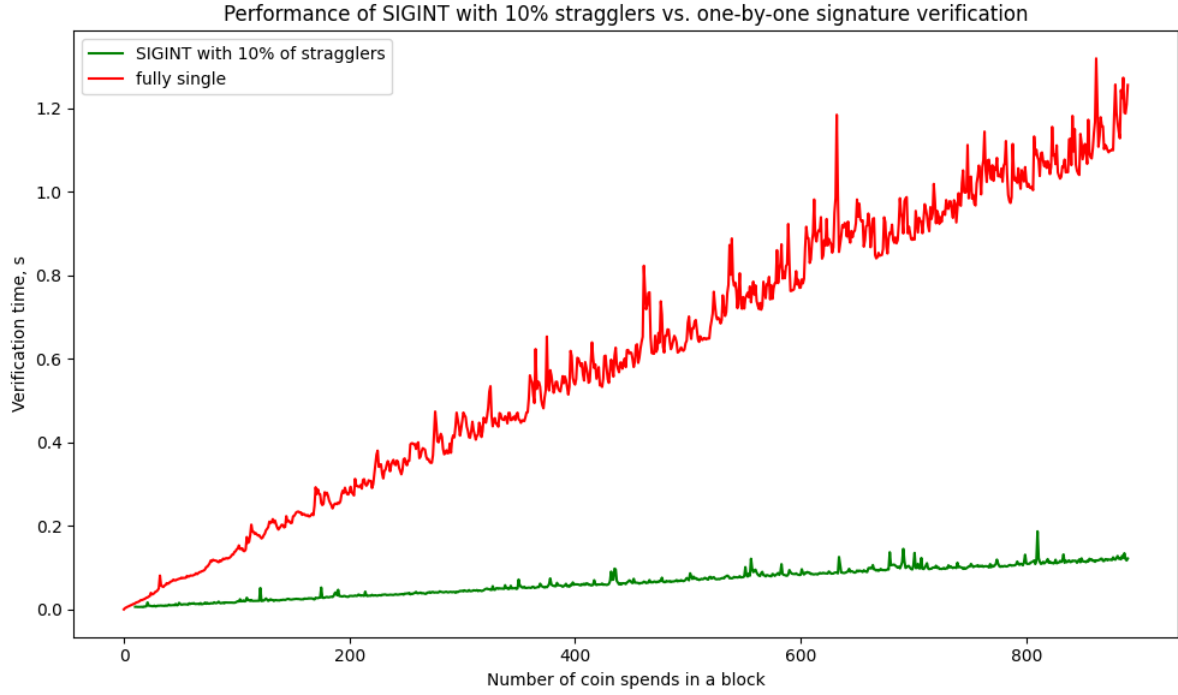


Figure 14: SIGINT validation performance with a fraction of stragglers.

Block verification times range from **0.005 seconds** per block up to **0.18 seconds** per block. It is clear that even with 10% of stragglers, SIGINT performance is still impressive compared to one-to-one signature validation. The result is that any block size can be chosen to be the optimal one, but to minimize the up-tick in noise, blocks of up to **300 coin spends** should be prioritized.

All of the results in this section helped to implement the SIGINT protocol inside the blockchain and it was observed that some parameters may be more optimal than others. Essentially, it helped to estimate the approximate behavior of such validation method in Chia blockchain and it was decided to furtherly observe how this approximation generalizes to the blockchain itself.

4.9 Modified Chia blockchain block validation metrics

After executing all previous tests and selecting optimal parameters, a simplified SIGINT verification protocol was established inside of Chia blockchain (as explained in section 3), which enabled the possibility to come back to syncing the blockchain under assumptions that: (1) all blocks contains only one fully aggregated multi-signature of all coin spends in the block and (2) there are no straggler signers in that block. This assumption can be called a *perfect SIGINT scenario*.

The *perfect SIGINT scenario* essentially enables all blocks to take full advantage of fast verification. To evaluate the impact it would have on Chia blockchain syncing performance, same benchmarking was carried out as in section 4.1 by syncing the full node up to **1017931 out of 3775228 blocks** and comparing the measurements between the classic Chia blockchain and perfect scenario SIGINT-enabled blockchain.

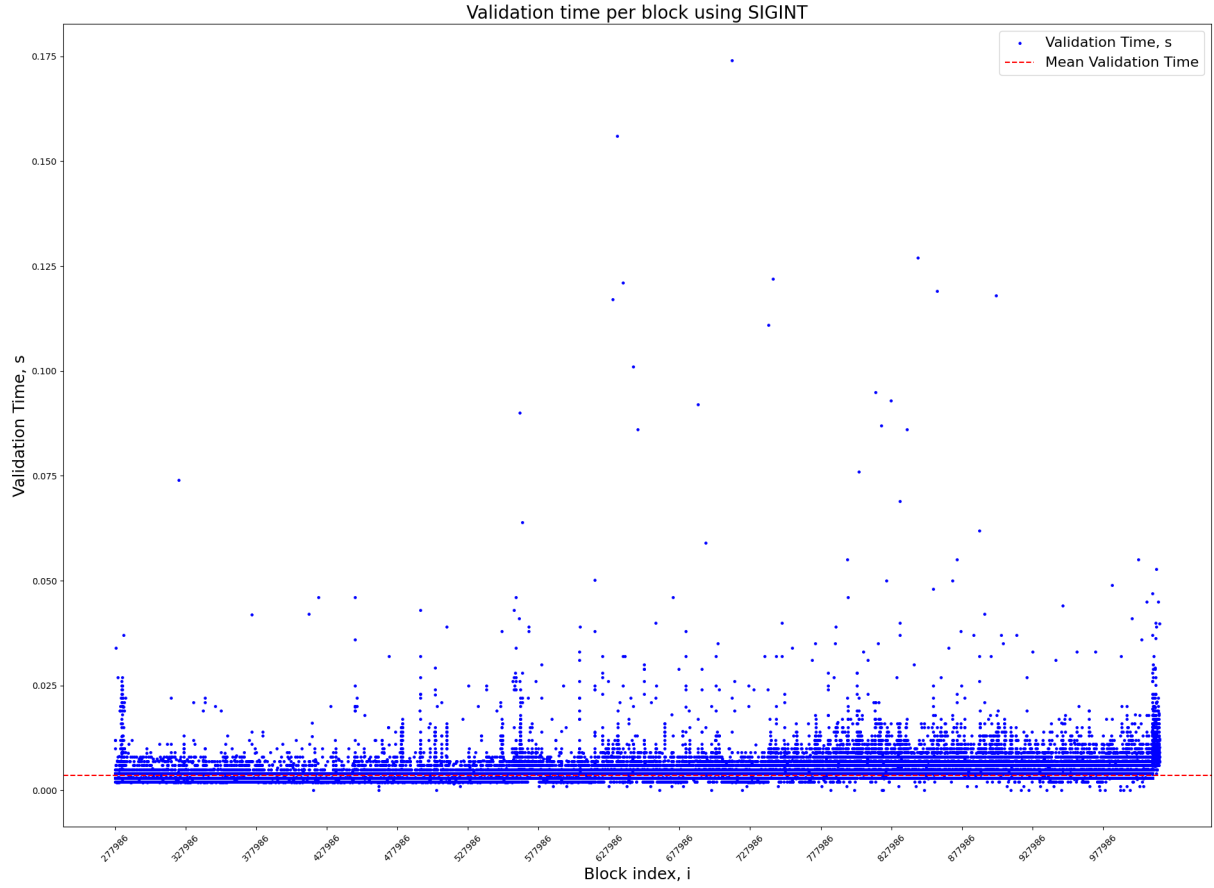


Figure 15: Validation time in a *perfect SIGINT* scenario

The new result on the measure of average time spent per block to validate its signatures is **0.004 seconds** per block with a maximal time of **0.17 seconds** per block. On average, **0.163 seconds** are saved per one non-empty block validation resulting in a **42 times** faster validation process. With the new SIGINT-enabled blockchain the same syncing process as performed initially would take **14 hours and 40 minutes** instead of **26 hours and 21 minutes** for original Chia blockchain.

One can observe that SIGINT certainly enables the blockchain to be more efficient than it originally is. Even though to evaluate the exact behavior extensive real-life tests would need to be performed, these benchmarks still provide valuable insights for the research of SIGINT protocol.

5 Conclusion

5.1 Project Outcome

The successful implementation of integrating SIGINT within a UTXO-like blockchain has demonstrated its feasibility.

Chia's unique approach to UTXO transactions forced us to reiterate over the aggregator implementation multiple times due to the complex design behind coins spends with respect to other UTXO systems like Bitcoin.

Chia also proved to be a flexible blockchain for swiftly implementing and testing changes. Its Python-based architecture minimizes recompilation times and provides the freedom to modify components without the limitations of a strongly typed system.

5.2 Final Considerations

Integrating SIGINT in a real-life system becomes significantly harder when it requires to integrate BLS signatures into the underlying system. If on the other hand, the existing infrastructure already supports BLS signatures, the integration process becomes relatively straightforward.

An additional challenge to face when considering the integration of SIGINT capabilities into an existing blockchain project is the task of convincing the community to adopt a system that requires significant modifications to the execution layer. The community's agreement and consensus are crucial for the successful implementation of any changes in a decentralized network.

Developing a SIGINT-native blockchain project, such as Chop Chop [2] has the potential to be more successful in terms of integrating SIGINT functionalities seamlessly. By designing the blockchain from the ground up with SIGINT in mind, the system can be built to accommodate and support the necessary features without compromising on performance or security.

A promising middle ground compromise for the future involves leveraging SIGINT to establish a gas factory within the smart contract layer of an existing blockchain. In the context of Ethereum, proposals like EIP-2537 [3] propose incorporating BLS Precompiles, which facilitate the consolidation of numerous transactions from diverse users into batches through the concept of account abstraction. This approach enables the creation of a gas factory, where a smart wallet serves as the sender on behalf of multiple users, thereby optimizing gas costs and acting as an onchain aggregator.

5.3 Possible Future Improvements

As mentioned in the implementation section many design and structural simplifications as well as assumptions were made to respect the time and depth constraints of the project. Hence, the following points are left as a future reference for continued exploration of SIGINT interactive protocol.

- Communication between the aggregator and the node should fully happen over the SSL secured JSON-RPC native protocol.
- All the placeholder verifications functions should actually make sure of the correctness of the transactions received. Ideally this should be done by referencing the same code used in the blockchain to verify blocks/transactions by eventually modularizing it when necessary to avoid code duplication.
- Gas costs for the transaction has not been taken into account in the sandboxed environment used for testing. Aggregating transaction could even help saving gas since multiple senders can pay together.
- Testing should be executed in a real blockchain-dedicated environment that is efficient in executing blockchain operations.
- SIGINT interactive protocol part should be extensively tested with a system of multiple clients, aggregators and validators that would continuously send and generate new payloads. Stress-testing and safety-testing could also be performed to remark the issues and bottlenecks.

- Further in-depth collaboration with Chia blockchain developer team may be established to work on natively implementing SIGINT protocol in the blockchain.

5.4 Attributions

A sentence of gratitude to our supportive and helpful project supervisor Pierre-Louis. It was a pleasure to perform research on a state-of-art invention and to explore possible ways of implementing it.

We would also like to mention the developers of the Chia network since they have been remarkably accessible and provided valuable assistance in addressing our inquiries regarding the specifics of the protocol.

References

- [1] Martina Camaioni, Rachid Guerraoui, Matteo Monti, Pierre-Louis Roman, Manuel Vidigueira, and Gauthier Voron. Rapidly Verifiable Aggregate Signatures. Patent pending, patent number: EP22197596.4.
- [2] Martina Camaioni, Rachid Guerraoui, Matteo Monti, Pierre-Louis Roman, Manuel Vidigueira, and Gauthier Voron. Chop chop: Byzantine atomic broadcast to the network limit, 2023.
- [3] Alex Vlasov and Kelly Olson. EIP-2537: Precompile for BLS12-381 curve operations [DRAFT]. Ethereum Improvement Proposals, February 2020.